

Writing `+42` (or `++42`) seems just as silly as `--42`, so why does Python even have this unary operator? It exists only to complement the `-` operator if you need to overload these operators for your own classes. (That’s a lot of terms you might not be familiar with! You’ll learn more about operator overloading in Chapter 17.)

The `+` and `-` unary operators are only valid when in front of a Python value, not after it. Although `spam++` and `spam--` might be legal code in C++ or JavaScript, they produce syntax errors in Python:

```
>>> spam++
      File "<stdin>", line 1
        spam++
          ^
SyntaxError: invalid syntax
```

Python doesn’t have increment and decrement operators. A quirk of the language syntax merely makes it seem like it does.

All of Nothing

The `all()` built-in function accepts a sequence value, such as a list, and returns `True` if all the values in that sequence are “truthy.” It returns `False` if one or more values are “falsey.” You can think of the function call `all([False, True, True])` as equivalent to the expression `False and True and True`.

You can use `all()` in conjunction with list comprehensions to first create a list of Boolean values based on another list and then evaluate their collective value. For example, enter the following into the interactive shell:

```
>>> spam = [67, 39, 20, 55, 13, 45, 44]
>>> [i > 42 for i in spam]
[True, False, False, True, False, True, True]
>>> all([i > 42 for i in spam])
False
>>> eggs = [43, 44, 45, 46]
>>> all([i > 42 for i in eggs])
True
```
